



Welcome to the Algorithmic & DSA Curriculum

This roadmap is designed to take software engineers and competitive programmers from basic coding language syntax to advanced algorithmic problem solving. The content is structured hierarchically to guarantee step-by-step progress. It covers programming fundamentals, classic data structures, complex optimization strategies (Dynamic Programming, Greedy), advanced range-query trees, and competitive programming invariants. It is suitable for preparing for product companies, LeetCode, Codeforces, and Google/Meta style interviews.

Table of Stages / Content Map:

- Stage 0 — Programming Fundamentals & Complexity Analysis
- Stage 1 — Arrays & Strings (Traversals, Two Pointers, Sliding Window)
- Stage 2 — Linked Lists (Singly, Doubly, Circular, Cycles)
- Stage 3 — Stack & Queue (Dequeues, Monotonic structures, Range Maximums)
- Stage 4 — Hashing & Sets (Frequency maps, Collisions)

- Stage 5 — Recursion & Backtracking (Permutations, N-Queens, Sudoku)
- Stage 6 — Binary Trees (Traversals, BFS, DFS, Ancestors, Serializers)
- Stage 7 — Binary Search Trees (Properties, Operations, Balancing)
- Stage 8 — Heaps & Priority Queues (Median of Stream, Top K)
- Stage 9 — Graphs (BFS, DFS, Cycles, Topological Sorts)
- Stage 10 — Shortest Paths (Dijkstra, Bellman-Ford, Floyd-Warshall)
- Stage 11 — Minimum Spanning Trees & Disjoint Set Union (DSU)
- Stage 12 — Dynamic Programming (Memoization, Tabulation, 18 Patterns)
- Stage 13 — Greedy Algorithms (Scheduling, Exchange arguments)
- Stage 14 — Advanced Data Structures & Algorithms (Segment Trees, Trie, KMP)
- Stage 15 — Bit Manipulation (AND/OR/XOR operations, Masks)
- Stage 16 — Mathematical Algorithms (Modular inverse, Sieve, Exponentiation)
- Stage 17 — Competitive Programming Techniques (Coordinate compression, Sweep-line)
- Stage 18 — 18 Problem-Solving Pattern Templates
- Stage 19 — Practice Roadmap (Level 1 to Level 5 Problem Sheets)
- Stage 20 — Dedicated Interview Preparation Tracks

Stage 0 — Programming Fundamentals & Complexity

Master the fundamentals of your chosen language (Python for implementation templates) and learn the asymptotic notation needed to evaluate algorithm efficiency.

Language Syntax Basics

[MUST KNOW] Variables, basic data types (integers, floats, strings, booleans), standard input/output, conditional logic (if-elif-else), loops (for, while), and functions.

Recursion Fundamentals

[MUST KNOW] A function calling itself to break down problems. Focus on base cases, state transitions, recursion stack frames, and preventing stack overflows.

Object-Oriented Programming (OOP)

[SHOULD KNOW] Classes, object instantiations, attribute state, and constructor methods. Crucial for custom tree/graph node designs.

Python Built-Ins & Collections

[MUST KNOW] Operations on lists, sets, and dictionaries. Master methods like list appending/slicing, set hashing/lookups, and dictionary get/default values.

Iterators & Lambdas

[SHOULD KNOW] Generating ranges, traversing custom collections using iterators, and using anonymous functions (lambdas) for custom sorting keys.

Basic Exception Handling

[MUST KNOW] Using try-except blocks to catch division errors, key errors, and index out of bounds to write resilient code.

Sorting

[MUST KNOW] Standard sorting operations, custom key sorting, comparator functions, and understanding Python's Timsort complexity ($O(N \log N)$).

Asymptotic Complexity Analysis

Evaluate algorithm runtime and memory usage using asymptotic bounds. Avoid raw operations counting; focus on bounding scaling limits.

Big-O Notation (O)

[MUST KNOW] Represents the asymptotic upper bound of growth. Used to analyze worst-case behavior.

Big-Theta (Theta)

[SHOULD KNOW] Represents the tight asymptotic bound. Growth is bounded both above and below.

Big-Omega (Omega)

[SHOULD KNOW] Represents the asymptotic lower bound. Used to analyze best-case execution bounds.

Common Complexity Patterns Reference Table

Time Complexity	Operation Scaling Type	Common Algorithmic Pattern Examples
$O(1)$	Constant	Hash table lookups, array indexing, mathematical formulas, push/pop in stack.
$O(\log N)$	Logarithmic	Binary search, heap operations, modular exponentiation.
$O(N)$	Linear	Single loop traversals, linear search, prefix sums, queue operations.
$O(N \log N)$	Linearithmic	Merge sort, quicksort, heap sort, coordinate compression.
$O(N^2)$	Quadratic	Nested loops, bubble sort, brute force pairs search, 2D grid DP.
$O(2^N)$	Exponential	Generating all subsets of size N, recursive Fibonacci, backtracking combinations.
$O(N!)$	Factorial	Generating all permutations of an array, traveling salesperson TSP brute force.

Stage 1 — Arrays & Strings

Linear structures stored contiguously. Master traversal patterns and sliding window limits.

Contiguous Traversals

[MUST KNOW] Traversing arrays from left-to-right, right-to-left, or outer-to-inner. Core for basic indexing.

Two Pointers Technique

[MUST KNOW] Iterating using two index variables moving toward each other or moving at different speeds. Reduces $O(N^2)$ scans to $O(N)$.

Sliding Window (Fixed/Variable)

[MUST KNOW] Maintaining a subarray window that expands or contracts based on constraints. Focus on window resizing triggers.

Prefix Sums & Difference Arrays

[MUST KNOW] Prefix Sum: precomputes cumulative sum for $O(1)$ range queries. Difference Array: stores step differences to perform $O(1)$ range updates.

Frequency counting & Hashing

[MUST KNOW] Storing character/element occurrence counts inside a hash map or array to solve anagrams or duplicate check queries.

Sorting-Based Techniques

[MUST KNOW] Sorting inputs first to allow binary search, target pairs scanning, or interval merging.

Subarrays vs. Subsequences

[MUST KNOW] Subarray: contiguous slice of an array. Subsequence: elements in their original relative order but not necessarily contiguous.

Representative Problems

- Two Pointers: Target Sum (LeetCode 167 - Two Sum II - Input Array Is Sorted)
- Sliding Window: Longest Substring Without Repeating Characters (LeetCode 3)
- Prefix Sum: Range Sum Query (LeetCode 303 - Range Sum Query - Immutable)
- Difference Array: Range Addition (LeetCode 370)
- Frequency Map: Valid Anagram (LeetCode 242)

Stage 2 — Linked Lists

Dynamic node chains linked via memory pointers. Master pointer manipulations and node linkages.

Singly Linked Lists

[MUST KNOW] Nodes containing data and a 'next' pointer. Master basic node insertion and deletion.

Doubly Linked Lists

[MUST KNOW] Nodes containing 'next' and 'prev' pointers, allowing bidirectional traversals. Crucial for LRU cache implementations.

Circular Linked Lists

[SHOULD KNOW] Tail node points back to the head node. Used in round-robin scheduling algorithms.

Fast & Slow Pointers

[MUST KNOW] Two pointers traversal at rates of 2x and 1x. Used to find list midpoints and detect cycles.

Linked List Reversal

[MUST KNOW] Reversing link directions iteratively or recursively using pointer swapping.

Cycle Detection & Floyd's Algorithm

[MUST KNOW] Fast and slow pointers collide in a cycle loop. Locate the loop starting node using Floyd's Tortoise and Hare.

Merging & Intersecting lists

[MUST KNOW] Merging two sorted lists ($O(N)$); identifying list intersection nodes using length diffs or dual pointers.

Stage 3 — Stack & Queue

Linear containers following strict insertion and retrieval access bounds.

Stack (LIFO)

[MUST KNOW] Last In, First Out. Standard operations (push, pop, peek). Master recursion simulation and undo operations.

Queue (FIFO) & Deque

[MUST KNOW] First In, First Out. Deque (Double Ended Queue): supports push/pop at both ends. Essential for BFS tree/graph traversals.

Monotonic Stack

[MUST KNOW] Stack that maintains elements in strict increasing/decreasing order. Used for $O(N)$ next-greater-element checks.

Monotonic Queue

[MUST KNOW] Queue maintaining elements in sorted order. Used to track range maximums inside sliding windows.

Expression Evaluation

[SHOULD KNOW] Evaluating infix, prefix, and postfix notations. Master the Shunting-Yard algorithm for parsing operators.

Representative Problems

- Monotonic Stack: Next Greater Element (LeetCode 496), Largest Rectangle in Histogram (LeetCode 84)
- Monotonic Queue: Sliding Window Maximum (LeetCode 239)
- Parentheses parsing: Valid Parentheses (LeetCode 20), Min Add to Make Valid (LeetCode 921)

Stage 4 — Hashing

Using hash functions to map arbitrary-sized data inputs to fixed-size buckets for fast lookups.

Hash Table & Set

[MUST KNOW] Containers providing $O(1)$ average lookup, insert, and delete. Sets store keys; maps store key-value pairs.

Collision Intuition

[SHOULD KNOW] Understanding how keys map to the same bucket. Master Chaining (linked list buckets) and Open Addressing (linear/quadratic probing).

Frequency Maps

[MUST KNOW] Tracking element counts to locate duplicates, find majority elements, and scan target occurrences.

Classic Hashing Problems

- Two Sum (LeetCode 1) - standard frequency map lookup
- Subarray Sum Equals K (LeetCode 560) - prefix sum + frequency map pattern
- Longest Consecutive Sequence (LeetCode 128) - hash set lookup

Stage 5 — Recursion & Backtracking

Systematic search of space options by constructing path states and backtracking on dead-ends.

Recursion Trees

[MUST KNOW] Visualizing execution branches. Each tree node represents a recursive call; tree depth determines space complexity.

Backtracking Paradigm

[MUST KNOW] Constructing search paths step-by-step. If a path violates problem constraints, back up and try the next branch.

Permutations & Combinations

[MUST KNOW] Generating permutations (ordering matters, size $N!$) and combinations/subsets (order ignores, size 2^N).

Constraint Satisfaction Search

[SHOULD KNOW] Placing elements on boards under strict limits. Classic problems: N-Queens, Sudoku solver.

How to Identify Backtracking Problems

A problem requires backtracking when: 1) The output demands listing ALL possible combinations, permutations, or configurations. 2) The solution bounds scale exponentially (e.g., $N \leq 15$, indicating $O(2^N)$ or $O(N!)$ is expected). 3) Decisions must be validated step-by-step with rollback capabilities.

Stage 6 — Binary Trees

Hierarchical structures where each node has at most two child pointers. Master tree traversals.

Tree traversals (DFS)

[MUST KNOW] Depth-First Search. Traversing trees recursively: Preorder (Root-L-R), Inorder (L-Root-R), and Postorder (L-R-Root).

Tree traversals (BFS)

[MUST KNOW] Breadth-First Search. Traversal level-by-level using a Queue. Standard pattern for level-order printing.

Height & Depth

[MUST KNOW] Height: max path length from node to leaf. Depth: path length from root to node. Calculated recursively.

Diameter of Binary Tree

[MUST KNOW] Longest path between any two leaf nodes in the tree. May or may not pass through the root node.

Lowest Common Ancestor (LCA)

[MUST KNOW] The deepest node that is an ancestor to two target nodes. Essential recursive traversal check.

Tree Serialization

[SHOULD KNOW] Converting binary tree structure to a flat string representation (e.g., string list with '#' for nulls) and rebuilding it.

Stage 7 — Binary Search Trees (BST)

A binary tree where left child values are less than root, and right child values are greater.

BST Properties

[MUST KNOW] Left subtree values < node value < right subtree values. Crucial point: Inorder traversal outputs sorted values.

BST Operations

[MUST KNOW] Search, Insert, and Delete. Delete requires handling three cases: leaf node, one-child node, and two-children node.

BST Validation

[MUST KNOW] Validating if a binary tree is a valid BST. Do not check local parent-child relations only; maintain range limits [min, max] recursively.

Kth Smallest/Largest

[MUST KNOW] Locating the Kth sorted element. Perform Inorder traversal to count items, or track subtree sizes if dynamic.

Stage 8 — Heaps & Priority Queues

Complete binary trees that satisfy the heap property, providing fast access to maximum/minimum elements.

Heap Structure & Heapify

[MUST KNOW] Min Heap (parent $\leq$ children) and Max Heap (parent $\geq$ children). Heapify: $O(N)$ array-to-heap conversion. Push/Pop run in $O(\log N)$.

Priority Queues

[MUST KNOW] Abstract data structure providing fast min/max retrievals. Implemented natively in Python via `heapq`.

Top K Elements Problems

[MUST KNOW] Extracting K largest/smallest elements. Use a size-K min heap to track top values in $O(N \log K)$ time.

Median of Stream

[SHOULD KNOW] Tracking median dynamically in a continuous stream of numbers. Implement using two heaps: a max-heap for the lower half and a min-heap for the upper half.

Stage 9 — Graphs

Sets of vertices connected by edges. Master graph representations and traversals.

Graph Representation

[MUST KNOW] Adjacency Matrix: $O(V^2)$ grid (good for dense graphs). Adjacency List: list of edge arrays (best for sparse graphs, uses $O(V+E)$ memory).

Graph DFS & BFS

[MUST KNOW] BFS: level-order scan using Queue. DFS: recursive depth exploration using Call Stack. Must track visited vertices to prevent infinite loops.

Cycle Detection

[MUST KNOW] Undirected: check if visited neighbor is not parent. Directed: track recursion stack state (detect back edges).

Topological Sorting

[MUST KNOW] Linear ordering of vertices in a Directed Acyclic Graph (DAG) such that for edge $u-v$, u comes before v . Use Kahn's BFS (indegree counting) or DFS.

Bipartite Graphs

[SHOULD KNOW] Check if graph vertices can be colored using two colors such that no adjacent vertices share a color. Test using BFS/DFS coloring.

Stage 10 — Shortest Path Algorithms

Algorithmic frameworks designed to locate paths of minimal weights between vertices.

1. Dijkstra's Algorithm

[MUST KNOW] Intuition: Greedy search that expands the shortest known paths from a source node.

Algorithm: Maintains distances array. Uses Min-Priority Queue to extract unvisited node with minimum distance, relaxes neighbors, and repeats.

Time Complexity: $O((V + E) \log V)$.

Space Complexity: $O(V + E)$ (to store graph and priority queue).

When to use: Single-source shortest paths on graphs with non-negative edge weights.

When NOT to use: Graphs containing negative edge weights (fails to converge).

2. Bellman-Ford Algorithm

[SHOULD KNOW] Intuition: Dynamic programming approach that relaxes all edges $V-1$ times.

Algorithm: Loops $V-1$ times; in each iteration, loops over all edges to update min distance. A V -th iteration detects negative cycles.

Time Complexity: $O(V E)$.

Space Complexity: $O(V)$.

When to use: Single-source shortest paths with negative weights, or when negative cycle detection is required.

When NOT to use: Large graphs where $O(V E)$ is computationally prohibitive.

3. Floyd-Warshall Algorithm

[SHOULD KNOW] Intuition: Compares path transitions via an intermediate vertex k .

Algorithm: 3-nested loops: for each intermediate k , check if path $u-v$ can be shortened via k : $\text{grid}[i][j] = \min(\text{grid}[i][j], \text{grid}[i][k] + \text{grid}[k][j])$.

Time Complexity: $O(V^3)$.

Space Complexity: $O(V^2)$.

When to use: All-pairs shortest paths on small graphs ($V \leq 400$).

When NOT to use: Large graphs where cubic time is too slow.

4. 0-1 BFS Algorithm

[MUST KNOW] Intuition: Shortest path on graphs with edge weights restricted to 0 and 1.

Algorithm: Deque traversal. If weight is 0, append to front; if 1, append to back. Always extracts minimum distance first.

Time Complexity: $O(V + E)$.

Space Complexity: $O(V)$.

When to use: Shortest paths where edge transitions have binary cost (0 or 1).

When NOT to use: Graphs with fractional or arbitrary edge weights.

Stage 11 — Minimum Spanning Tree (MST) & DSU

Connecting all vertices in a graph with the minimum possible total edge weight without cycles.

Prim's Algorithm

[MUST KNOW] Greedy node-centric algorithm. Starts with a node and repeatedly appends the cheapest edge that connects a node in the tree to a node outside it. Uses Min Heap.

Kruskal's Algorithm

[MUST KNOW] Greedy edge-centric algorithm. Sorts all edges by weight, and repeatedly adds the cheapest edge if it doesn't create a cycle. Cycles are checked using DSU.

Disjoint Set Union (DSU)

[MUST KNOW] Data structure representing partition elements. Standard operations: Find (returns root) and Union (joins sets).

Union-Find Optimizations

[MUST KNOW] Path Compression (flattens tree structure to $O(1)$ during find operations) and Union by Rank/Size (attaches smaller trees to larger ones, keeping depth low).

Stage 12 — Dynamic Programming (DP) Roadmap

DP optimizes recursion by saving solutions to overlapping subproblems rather than recalculating them.

DP Core Concept

[MUST KNOW] Applicable when a problem displays: 1) Overlapping Subproblems (subproblems recalculate repeatedly), 2) Optimal Substructure (global optimal uses local subproblem optimals).

Memoization vs. Tabulation

[MUST KNOW] Memoization: Top-down recursion caching. Tabulation: Bottom-up iterative table resolution. Tabulation is generally faster but memoization is easier to write.

State & Transition

[MUST KNOW] State: Variables describing subproblem inputs. Transition: Formula connecting the current state to sub-states (e.g., $dp[i] = dp[i-1] + dp[i-2]$).

Space Optimization

[SHOULD KNOW] Reducing storage from $O(N)$ or $O(N^2)$ to $O(1)$ or $O(W)$ by keeping only the last few rows/elements needed for transitions.

18 Key DP Patterns & Recognition Strategies

1D DP (Linear State)

[MUST KNOW] State is a single parameter representing array index. Transition uses previous indices (e.g., Climbing Stairs).

2D DP / Grid DP

[MUST KNOW] State is index coordinates (i, j) on a grid. Transitions allow moving down/right (e.g., Unique Paths).

0/1 Knapsack

[MUST KNOW] State represents choice of item i and remaining weight w . Transition: include item vs. exclude item.

Unbounded Knapsack

[MUST KNOW] Same as 0/1, but items can be selected infinitely. Transition updates weight but keeps index constant.

Subsequence DP / LIS

[MUST KNOW] Longest Increasing Subsequence. State tracks indices i and j . Transition: $dp[i] = \max(dp[j] + 1)$ for all $j < i$ where $arr[j] < arr[i]$.

String DP / LCS

[MUST KNOW] Longest Common Subsequence. State tracks string lengths (i, j) . Transition: match $(1 + dp[i-1][j-1])$ vs. mismatch $(\max(dp[i-1][j], dp[i][j-1]))$.

Interval DP

[SHOULD KNOW] State represents range boundaries $[left, right]$. Solve sub-intervals first (e.g., Matrix Chain Multiplication).

Partition DP

[SHOULD KNOW] Partitioning array into K subsegments to minimize/maximize costs. Transition loops over possible partition cuts.

Tree DP

[ADVANCED] Performing DP on tree nodes. State represents node selection (e.g., selected vs. not selected in House Robber III).

Bitmask DP

[ADVANCED] State uses an integer mask (binary representation) to track visited sub-states (e.g., Traveling Salesperson TSP).

Digit DP

[ADVANCED] Counting count digits satisfying a property in a range $[L, R]$. State tracks index, digit limit, and condition constraints.

Stage 13 — Greedy Algorithms

Making locally optimal choices at each step, hoping it leads to a global optimum.

Greedy Intuition

[MUST KNOW] Choose the best local choice without looking back. Works only if local choices never rule out global optimals.

Exchange Argument

[SHOULD KNOW] Mathematical proof strategy to prove greedy correctness: assume another optimal ordering exists, swap elements, and show target cost doesn't decay.

Activity Selection / Interval Scheduling

[MUST KNOW] Given tasks with start/end times, select maximum tasks. Greedy strategy: sort by finish times, pick earliest finishing tasks first.

Fractional Knapsack

[MUST KNOW] Items can be split. Greedy strategy: sort by value/weight ratio, select highest ratios first.

Huffman Coding

[SHOULD KNOW] Greedy data compression. Build a tree by repeatedly combining the two least-frequent character nodes.

Stage 14 — Advanced Algorithms

Advanced data structures for range queries, prefix matching, and string patterns.

Trie (Prefix Tree)

[MUST KNOW] Node tree storing characters. Used to lookup word prefixes and perform search auto-completes in $O(\text{Length})$ time.

Segment Tree

[ADVANCED] Balanced binary tree representing range intervals. Perform range queries (Sum/Min/Max) and point updates in $O(\log N)$ time.

Fenwick Tree (BIT)

[ADVANCED] Binary Indexed Tree. Much faster and easier to code than Segment Trees; supports range sum queries and updates in $O(\log N)$.

Lazy Propagation

[ADVANCED] Deferring updates inside Segment Trees by buffering additions at parent nodes, reducing range update complexity from $O(N)$ to $O(\log N)$.

String Matching: KMP Algorithm

[ADVANCED] Knuth-Morris-Pratt. Avoids resetting scanning pointers using a prefix table (LPS array), finding string matches in $O(N+M)$ time.

String Hashing & Rabin-Karp

[SHOULD KNOW] Calculates sliding hash values (Rolling Hash) to find pattern matches in linear time.

Stage 15 — Bit Manipulation

Bitwise operations on integers, maximizing execution speed.

Bitwise Operators

[MUST KNOW] AND (&), OR (|), XOR (^), NOT (~), and shifts (<<, >>).

Basic Bit Operations

[MUST KNOW] Set bit: $x \mid (1 \ll i)$. Unset bit: $x \& \sim(1 \ll i)$. Toggle bit: $x \wedge (1 \ll i)$. Check bit: $(x \gg i) \& 1$.

Power of Two & Count Bits

[MUST KNOW] Power of Two: $x \& (x - 1) == 0$. Kernighan's algorithm count set bits: repeatedly clear lowest set bit $x \&= x-1$.

XOR Tricks

[MUST KNOW] Properties: $x \wedge x = 0$, $x \wedge 0 = x$. Used to find non-duplicate elements in pairs arrays (LeetCode 136).

Stage 16 — Mathematical Algorithms

Number theory and combinatorics necessary for competitive programming limits.

Greatest Common Divisor (GCD)

[MUST KNOW] Euclidean algorithm: $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$. Runs in $O(\log(\min(a, b)))$ time.

Prime Numbers & Sieve

[MUST KNOW] Sieve of Eratosthenes: precomputes prime flags up to N in $O(N \log(\log N))$ time. Prime checks up to $\sqrt{N}$.

Modular Arithmetic & Exponentiation

[MUST KNOW] Modular addition, subtraction, multiplication. Binary Exponentiation: calculates $a^b \% \text{mod}$ in $O(\log b)$ time.

Modular Multiplicative Inverse

[SHOULD KNOW] Finding x such that $(a * x) \% \text{mod} == 1$. Uses Fermat's Little Theorem if mod is prime: $a^{(\text{mod}-2)} \% \text{mod}$.

Combinatorics (nCr)

[SHOULD KNOW] Calculating combinations. Precompute factorials and modular inverses to query $nCr \% \text{mod}$ in $O(1)$ time.

Stage 17 — Competitive Programming Techniques

Optimization patterns focused on beating testing limits and handling massive queries.

Coordinate Compression

[SHOULD KNOW] Mapping large, sparse coordinates values (e.g. up to 10^9) to index values from 0 to $N-1$ while preserving relative order.

Sweep Line Paradigm

[SHOULD KNOW] Iterating over events sorted by coordinates (e.g. interval merges, intersection queries). Tracks active state dynamically.

Meet in the Middle

[ADVANCED] Split dataset of size N into two halves of size $N/2$, search both halves independently, and merge outputs (reduces $O(2^N)$ to $O(2^{(N/2)})$).

Binary Search on Answer

[MUST KNOW] Applying binary search on the output value space $[\text{min_val}, \text{max_val}]$ instead of array indices, using a greedy helper to test validity.

Stage 18 — Problem-Solving Templates

Reusable algorithmic templates for the 18 core design patterns.

1. Two Pointers

How to Recognize: Input is a sorted array/list, and we need to find pairs matching a target sum or condition.

```
def two_pointers(arr, target):
    left, right = 0, len(arr) - 1
    while left < right:
        val = arr[left] + arr[right]
        if val == target: return left, right
        elif val < target: left += 1
        else: right -= 1
    return -1
```

2. Sliding Window (Variable Size)

How to Recognize: Find the contiguous subarray (longest/shortest) matching a sum or distinct characters constraint.

```
def sliding_window(arr, constraint):
    left, max_len = 0, 0
    state = {}
    for right in range(len(arr)):
        # update state with arr[right]
        while not valid(state):
            # remove arr[left] from state
            left += 1
        max_len = max(max_len, right - left + 1)
    return max_len
```

3. Binary Search Template

How to Recognize: Target query on a sorted lookup space, or finding minimum/maximum matching values.

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    ans = -1
    while low <= high:
        mid = (low + high) // 2
        if condition(arr[mid], target):
            ans = mid
            high = mid - 1 # search left
        else:
            low = mid + 1 # search right
    return ans
```

4. Breadth-First Search (BFS) Graph

How to Recognize: Shortest path on unweighted graphs, or level-order tree expansions.

```
from collections import deque
def bfs(graph, start):
    visited = {start}
    queue = deque([start])
    while queue:
        node = queue.popleft()
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

5. Backtracking Template

How to Recognize: Generate all subsets, combinations, permutations, or validate N-Queens puzzle configurations.

```
def backtrack(state, options, path, result):
    if is_solution(state):
        result.append(list(path))
        return
    for option in options:
        if valid(option, state):
            path.append(option)
            update(state, option)
            backtrack(state, options, path, result)
            path.pop() # rollback
            rollback(state, option)
```

6. Topological Sort (BFS Indegree)

How to Recognize: Schedule tasks under dependencies (e.g. Course Schedule), finding scheduling sequence.

```
from collections import deque
def topo_sort(num_nodes, graph):
    indegree = {i: 0 for i in range(num_nodes)}
    for u in graph:
        for v in graph[u]: indegree[v] += 1
    queue = deque([n for n in indegree if indegree[n] == 0])
    order = []
    while queue:
        u = queue.popleft()
        order.append(u)
        for v in graph[u]:
            indegree[v] -= 1
            if indegree[v] == 0: queue.append(v)
    return order if len(order) == num_nodes else []
```


Stage 19 — Practice Roadmap

A progressive, structured problem sheets to gauge validation competency.

Level 1 — Fundamentals (50 Problems)

[SHOULD KNOW] LeetCode Easy, AtCoder ABC (A-B). Focus on array loops, basic stacks, string processing, and $O(N)$ traversals. Expected Skills: basic syntax, recursion trees.

Level 2 — Intermediate (100 Problems)

[SHOULD KNOW] LeetCode Medium, Codeforces Div2 (A-B). Focus on two pointers, sliding windows, hashing maps, linked lists, and recursion. Expected Skills: $O(N)$ optimizations, DFS/BFS.

Level 3 — Advanced Patterns (150 Problems)

[SHOULD KNOW] LeetCode Medium/Hard, Codeforces Div2 (C). Focus on backtracking subsets, heaps, basic BSTs, standard graphs (BFS/DFS, Topological Sort), and basic DP. Expected Skills: recursion path tracking.

Level 4 — Optimization (200 Problems)

[SHOULD KNOW] LeetCode Hard, Codeforces Div2 (D-E). Focus on complex 2D DP, shortest path graphs (Dijkstra), MST trees, Trie matching, and Fenwick trees. Expected Skills: DP transitions.

Level 5 — Competitive Programming (Continuous Practice)

[SHOULD KNOW] Codeforces Div1 (A-C), AtCoder AGC. Focus on Range queries (Segment Trees), mathematical inversions, modular inverses, bitmask DP, and off-line queries. Expected Skills: CP optimizations.

Stage 20 — Dedicated Interview Tracks

Tailored tracks depending on target career aspirations and interview environments.

Product Companies (Meta/Google/Amazon)

[MUST KNOW] Focus Areas: Medium/Hard LeetCode problems (80% Medium, 20% Hard). Master Two Pointers, Trees, Graphs, BFS/DFS, and dynamic programming patterns.

Key Competency: Ability to identify the underlying pattern (e.g. Monotonic Stack) quickly and explain time/space complexities clearly under pressure.

Competitive Programming Track (Codeforces/AtCoder)

[ADVANCED] Focus Areas: Mathematical number theory, range trees, sweep line coordinates, segment trees, and offline queries.

Key Competency: Absolute speed, optimization under tight time limit constraints (e.g. $O(N \log N)$ limit for $N=2 \cdot 10^5$), and handling complex invariants.

General Software Engineering Interviews

[SHOULD KNOW] Focus Areas: LeetCode Easy to Medium. Solid implementation of classic stacks, queues, hash sets, binary trees, and basic recursion.

Key Competency: Readable code, clean variable names, explaining code flow, dry-running inputs, and handling basic edge cases.

Recommended Timeline Plans & Checklists

Structured timetables designed for algorithmic progression.

Duration	Target Focus	Weekly Milestones
30-Day Plan	Core Syntax & Lin. structures	Week 1-2: Complexity, Arrays, Two Pointers, Strings. Week 3: Stacks, Queues, Hash Maps. Week 4: Singly Lists, recursion basics, and Level 1 problems.
3-Month Plan	Non-Linear & Basic Graphs	Month 1: Arrays, Lists, Stacks. Month 2: Binary Trees, BST validations, Heaps. Month 3: Graph BFS/DFS, basic backtracking combinations, and Level 2 problems.
6-Month Plan	Optimization & Algorithms	Months 1-3: Non-Linear trees & Graphs. Month 4: Shortest paths, Prim's, DSU. Month 5: Dynamic Programming, Greedy selections. Month 6: Trie, bitwise algorithms, and Level 3 problems.
12-Month Plan	CP Specialist & Advanced structures	Months 1-6: Advanced DSA patterns. Months 7-9: Segment Trees, Fenwick, string KMP, math sieve. Months 10-11: Coordinate sweeps, offline queries, mock contests. Month 12: System case studies, interview revision.

Do Not Learn Everything at Once

To prevent cognitive overload, postpone these advanced topics until your core data structures (arrays, lists, trees, graphs) and optimization paradigms (basic DP, recursion, sorting) are solid: lazy propagation segment trees, string KMP, digit DP, bitmask DP, coordinate sweeps, and number theory modular multiplicative inverses.

Final DSA Competency Checklist

- ☐ Can analyze any recursive function's time and space complexity using a recursion tree.
- ☐ Knows when to use Two Pointers vs. Sliding Window vs. Prefix Sums.
- ☐ Can implement Singly and Doubly Linked List node deletions and reversals without syntax errors.
- ☐ Understand next-greater-element monotonic stack algorithms.
- ☐ Can validly check cycle detections in directed and undirected graphs.
- ☐ Can define DP states and establish transitions for Knapsack and Subsequence problems.
- ☐ Can package a graph topological sort using Kahn's BFS or DFS.